ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No.34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

Alliance School of Advanced Computing

Course Name: Graph Analysis

Course Code: E1ITC302

Assignment No: 3

Name of the Student: CHAVVA HARSHAVARDHAN REDDY
Registration Number: 2411021060907

A. Title of The Problem:

Min Cost to Connect All Points (LeetCode 1584)

B. Problem Statement in details:

- You are given an array points where points[i] = [xi, yi] represents the coordinates of a point on a 2D plane.
- The cost of connecting two points [xi, yi] and [xj, yj] is their Manhattan distance: |xi − xj| + |yi − yj|.
- Return the minimum total cost to connect all the points so that there is exactly one simple path between any two points.
- In graph terms, the points are the nodes of a complete weighted graph, and the task is to find the weight of its Minimum Spanning Tree (MST).

C. HackerRank/Leetcode Link of the problem (If applicable):

https://leetcode.com/problems/min-cost-to-connect-all-points/

D. Student's Github link of report and code upload, along with the readme file:

GITHUB link

E. Solution Steps/Algorithm:

The solution uses Prim's algorithm on the implicit complete graph. No edge list is built; the Manhattan distance is computed on demand.

1. **Initialize Arrays**
   Set up the distance and visited arrays.
   - minDist[i] = ∞ for every point, visited[i] = false
   - Start from point 0: minDist[0] = 0
   - totalCost = 0

2. **Select the Closest Unvisited Point**
   Pick the next point to add to the tree.
   - Scan all points for the unvisited one with the smallest minDist
   - Mark it as visited
   - Add its minDist value to totalCost

3. **Update Neighbour Distances**
   Relax the cost of connecting every remaining point.
   - For each unvisited point v, compute the Manhattan distance |x1 − x2| + |y1 − y2| to the chosen point u
   - If it is smaller than minDist[v], update minDist[v]

4. **Repeat for All Points**

   Grow the spanning tree one point at a time.

   - Repeat steps 2 and 3 exactly n times
   - Each round adds one new point, giving n − 1 edges in total

5. **Return the Result**

   Output the cost of the Minimum Spanning Tree.

   - totalCost is the minimum total cost to connect all points
   - Return totalCost

F. Code developed:

```
import java.util.Arrays;

class Solution {
    public int minCostConnectPoints(int[][] points) {
        int n = points.length;

        int[] minDist = new int[n];
        boolean[] visited = new boolean[n];

        Arrays.fill(minDist, Integer.MAX_VALUE);
        minDist[0] = 0;

        int totalCost = 0;

        for (int i = 0; i < n; i++) {
            int u = -1;

            // Find the unvisited point with minimum distance
            for (int j = 0; j < n; j++) {
                if (!visited[j] &&
                    (u == -1 || minDist[j] < minDist[u])) {
                    u = j;
                }
            }

            visited[u] = true;
            totalCost += minDist[u];

            // Update distances to the remaining points
            for (int v = 0; v < n; v++) {
                if (!visited[v]) {
                    int distance =
                        Math.abs(points[u][0] - points[v][0])
                        + Math.abs(points[u][1] - points[v][1]);

                    minDist[v] = Math.min(minDist[v], distance);
                }
            }
        }

        return totalCost;
    }
}
```

G. LeetCode test case passed Screenshot or Output screenshot (Mandatory):



H. Observation:

The solution applies Prim's algorithm to a complete graph whose edge weights are Manhattan distances. Starting from point 0, it repeatedly adds the cheapest unvisited point to the growing tree, so exactly n − 1 edges are chosen and the total is the MST cost.

Dry run for points = [[0,0],[2,2],[3,10],[5,2],[7,0]]; the expected output is 20.

Step	Chosen point u	minDist after update	Total cost
1	0 (start)	[0, 4, 13, 7, 7]	0
2	1	[0, 4, 9, 3, 7]	4
3	3	[0, 4, 9, 3, 4]	7
4	4	[0, 4, 9, 3, 4]	11
5	2	[0, 4, 9, 3, 4]	20

Complexity: Time $O(n^2)$, because each of the n rounds does one O(n) scan for the minimum and one O(n) update. Space O(n) for minDist and visited.

Because the graph is dense (about $n^2/2$ edges), this array-based Prim's is better suited than Kruskal's algorithm with sorting, which would need $O(n^2 \log n)$ time and $O(n^2)$ memory to store all edges. The code also avoids a priority queue, which keeps it simple and fast for the problem's limit of $n \leq 1000$.